Homework 11

Question 1 Answers

Part a

The seed is 43211. The initial assignment uses `sample(rep(1:2, length.out = 10))`, so each cluster starts with 5 observations.

```
set.seed(43211)
X <- matrix(rnorm(20), 10, 2, dimnames = list(NULL, c("x1", "x2")))
C0 <- sample(rep(1:2, length.out = 10))

means_1 <- rbind(colMeans(X[C0 == 1, ]), colMeans(X[C0 == 2, ]))
dist_1 <- sapply(1:2, function(k) rowSums(sweep(X, 2, means_1[k, ])^2))
C1 <- max.col(-dist_1, ties.method = "first")

print(C0)
```

```
 [1] 2 1 1 1 1 2 2 2 2 1
```

```
print(round(means_1, 4))
```

```
         x1      x2
[1,] 0.6221  0.1829
[2,] 0.0374 -0.4525
```

```
print(C1)
```

```
 [1] 1 2 1 1 2 1 2 2 2 2
```

```
cols <- c("#1F5A94", "#C84A16")
plot(X, pch = 16, col = cols[C1], xlab = "x1", ylab = "x2")
points(means_1, pch = 4, cex = 2, lwd = 2, col = cols)
legend("topleft", bty = "n", pch = 16, col = cols, legend = c("Cluster 1", "Cluster 2"))
```

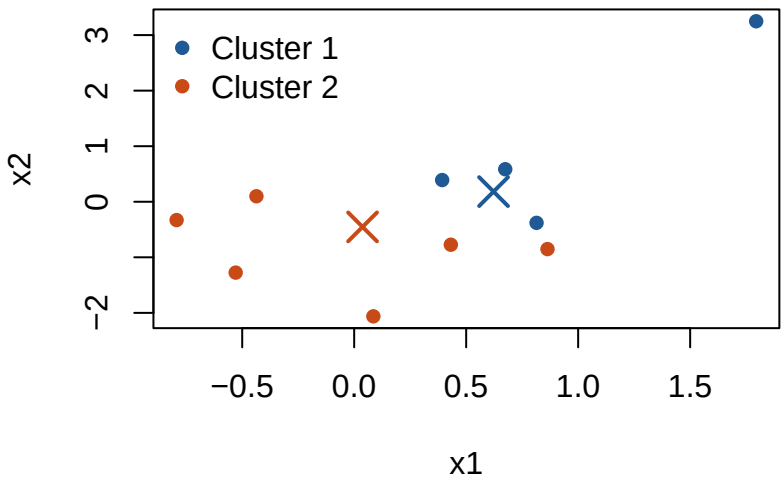


Figure 1: Observations colored by their assignments after one iteration, with the cluster means used for the assignment (crosses).

The initial assignment is $(2, 1, 1, 1, 1, 2, 2, 2, 2, 1)$. The cluster means are $m_1 = (0.622, 0.183)$ and $m_2 = (0.037, -0.453)$. After reassignment, the clusters are $(1, 2, 1, 1, 2, 1, 2, 2, 2, 2)$, with observations 1, 3, 4, and 6 in cluster 1.

Part b

```
my_kmeans <- function(X, K, init, max_iter = 100) {
  X <- as.matrix(X)
  C <- init
  means <- matrix(NA, K, ncol(X), dimnames = list(1:K, colnames(X)))

  # An empty cluster keeps its previous mean.
  update_means <- function(C, means) {
    for (k in 1:K) {
      if (any(C == k)) means[k, ] <- colMeans(X[C == k, , drop = FALSE])
    }
    means
  }

  for (iter in 1:max_iter) {
    means <- update_means(C, means)
    d <- sapply(1:K, function(k) rowSums(sweep(X, 2, means[k, ])^2))
    C_new <- max.col(-d, ties.method = "first")
    if (all(C_new == C)) break
    C <- C_new
  }

  means <- update_means(C, means)
```

```
  wss <- sum((X - means[C, , drop = FALSE])^2)
  list(cluster = C, means = means, wss = wss, iter = iter)
}
```

Part c

```
set.seed(1)
run_1 <- my_kmeans(X, 2, sample(rep(1:2, length.out = 10)))
set.seed(3)
run_2 <- my_kmeans(X, 2, sample(rep(1:2, length.out = 10)))

for (run in list(run_1, run_2)) {
  print(run$cluster)
  print(round(run$means, 4))
  print(c(wss = run$wss, iterations = run$iter))
}
```

```
 [1] 1 2 2 1 2 1 2 2 2 2
      x1      x2
1 0.9540  1.4086
2 0.0622 -0.7963
       wss iterations
  11.90389    3.00000
 [1] 2 1 1 2 1 2 1 1 1 1
      x1      x2
1 0.0622 -0.7963
2 0.9540  1.4086
       wss iterations
  11.90389    3.00000
```

```
par(mfrow = c(1, 2))
for (r in 1:2) {
  run <- list(run_1, run_2)[[r]]
  plot(X, pch = 16, col = cols[run$cluster], xlab = "x1", ylab = "x2",
       main = paste0("Run ", r, " (seed ", c(1, 3)[r], ")"))
  points(run$means, pch = 4, cex = 2, lwd = 2, col = cols)
}
```

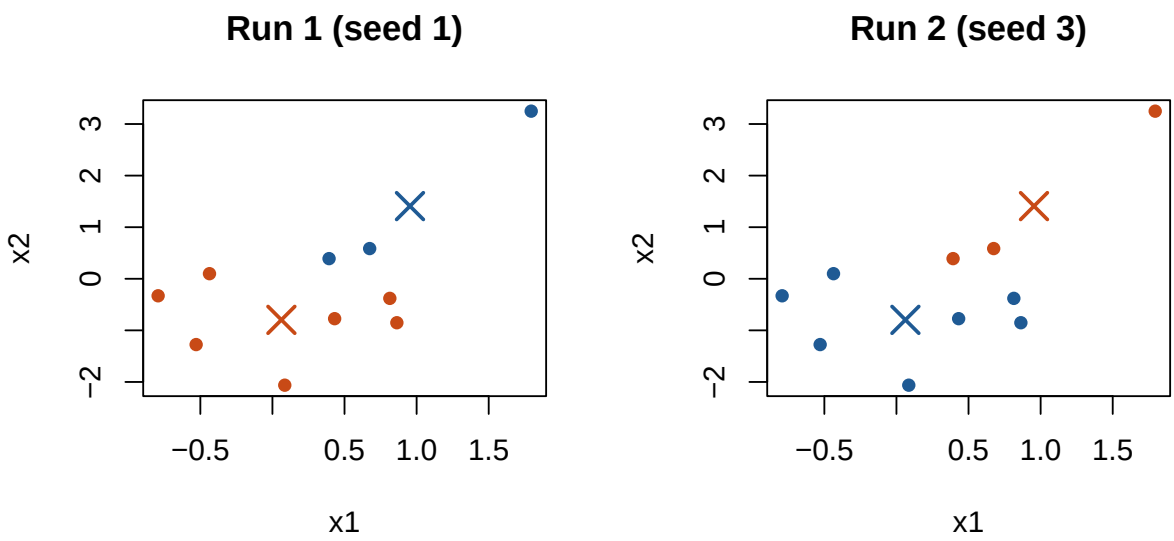


Figure 2: Final K-means clusters from two random initial assignments, with final cluster means (crosses).

Both runs converge in 3 iterations to the same partition: observations 1, 4, and 6 form one cluster with mean (0.954, 1.409), and the other 7 observations form the second with mean (0.062, −0.796). The within-cluster sum of squares is 11.904 for both. Only the labels are swapped, and they are arbitrary. K-means is guaranteed only to reach a local minimum of the within-cluster sum of squares, so different starts can give different solutions. With 10 points and a clearly separated upper-right group, both starts reached the same solution here. In general, K-means should be run from several starts, keeping the fit with the lowest objective.

Question 2 Answers

Part a

The image is a pixel-art diamond, converted from WebP to JPEG and saved as `data/diamond.jpg`. It is resized by nearest-neighbor sampling of rows and columns.

```
library(jpeg)

img_full <- readJPEG("data/diamond.jpg")
rows <- round(seq(1, dim(img_full)[1], length.out = 200))
cols_img <- round(seq(1, dim(img_full)[2], length.out = 200))
img <- img_full[rows, cols_img, ]

pixels <- matrix(img, ncol = 3, dimnames = list(NULL, c("red", "green", "blue")))

print(dim(img_full))
```

```
[1] 256 256   3
```

```
print(dim(img))
```

```
[1] 200 200   3
```

```r
print(dim(pixels))
```

```
[1] 40000     3
```

```r
show_image <- function(a, main = "") {
  par(mar = c(0, 0, 1.5, 0))
  plot(as.raster(a))
  title(main)
}
show_image(img)
```



Figure 3: Original image after resizing.

The original file is 256×256; the resized image is 200×200 with 3 channels, giving a $40{,}000 \times 3$ pixel matrix.

Part b

```r
set.seed(43212)
K_grid <- c(2, 3, 5, 10)
fits <- lapply(K_grid, function(K) {
  my_kmeans(pixels, K, sample(rep(1:K, length.out = nrow(pixels))), max_iter = 300)
})
names(fits) <- K_grid

for (K in as.character(K_grid)) {
  cat("K =", K, "\n")
  print(round(fits[[K]]$means, 3))
  print(c(wss = fits[[K]]$wss, iterations = fits[[K]]$iter))
}
```

```
K = 2
    red green  blue
1 0.966 0.996 0.988
2 0.155 0.514 0.505
       wss iterations
```

```
   1232.72      13.00
K = 3
    red green  blue
1 0.287 0.744 0.700
2 0.982 0.998 0.994
3 0.077 0.334 0.345
       wss iterations
  489.3784    33.0000
K = 5
    red green  blue
1 0.069 0.311 0.329
2 0.435 0.896 0.809
3 0.997 0.999 0.999
4 0.195 0.641 0.618
5 0.698 0.974 0.894
       wss iterations
  177.6919    28.0000
K = 10
     red green  blue
1  0.644 0.972 0.889
2  0.018 0.210 0.245
3  0.189 0.660 0.642
4  0.868 0.990 0.962
5  0.478 0.906 0.807
6  0.116 0.417 0.412
7  0.361 0.897 0.822
8  0.999 1.000 1.000
9  0.717 0.975 0.889
10 0.359 0.665 0.628
       wss iterations
  71.32279   53.00000
```

```r
par(mfrow = c(2, 2))
for (K in as.character(K_grid)) {
  fit <- fits[[K]]
  compressed <- array(fit$means[fit$cluster, ], dim = dim(img))
  show_image(compressed, main = paste("K =", K))
}
```

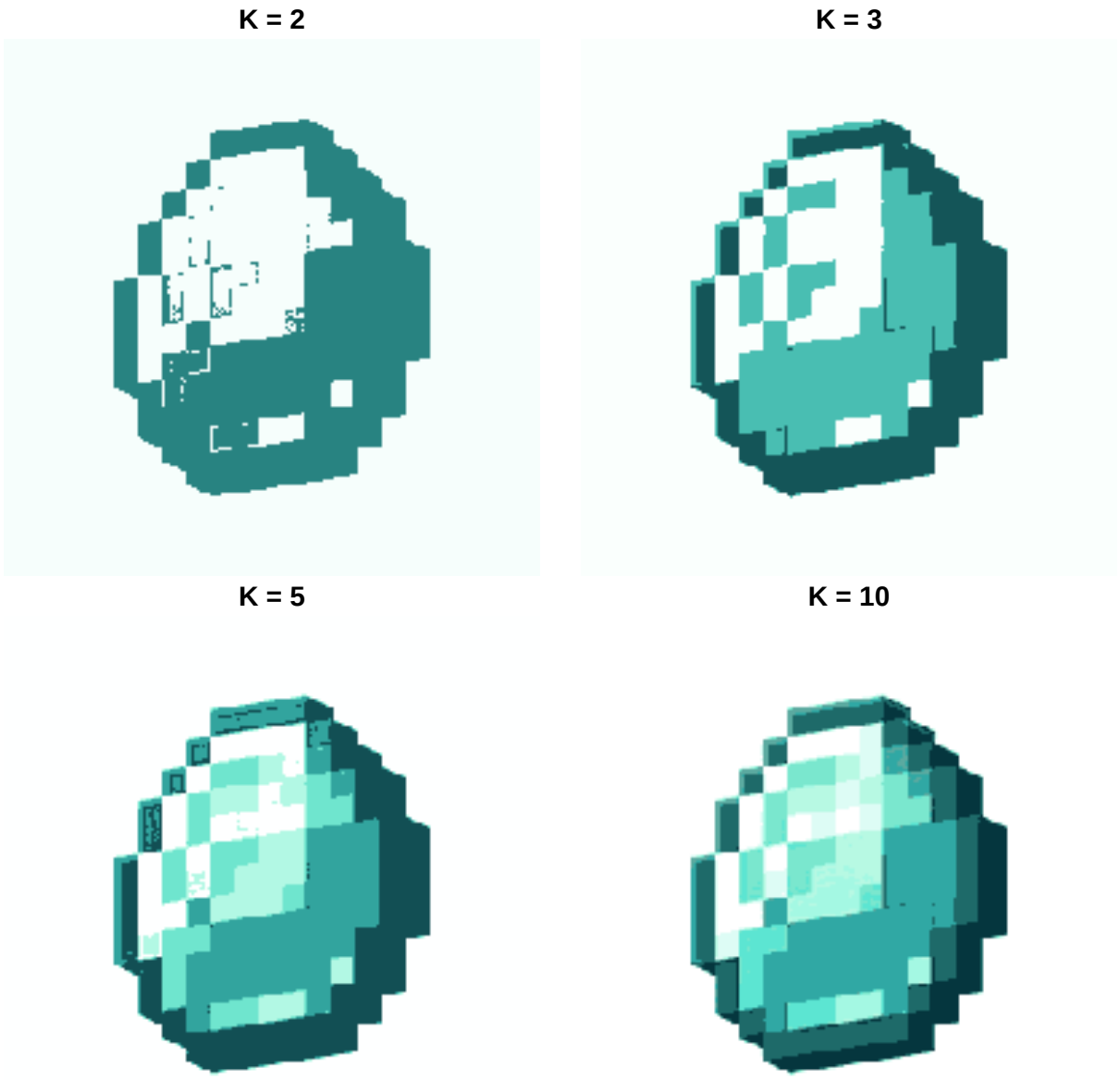


Figure 4: Compressed images, each pixel replaced by its cluster mean (RGB values on the common 0-1 scale).

Every fit converged before the 300-iteration limit. Since all RGB values stay in $[0, 1]$, `as.raster` maps each fit to colors on the same scale with no rescaling.

Part c

The within-cluster sums of squares are 1232.7, 489.4, 177.7, and 71.3 for $K = 2, 3, 5, 10$. $K = 2$ separates only the white background from one teal, and the diamond's highlights merge into the background. $K = 3$ recovers the outline and the light and dark sides. $K = 5$ adds the middle shades that give the facets their shape, and it looks close to the original. $K = 10$ lowers the objective further, but its extra clusters mostly split shades that are already similar, such as near-white values of 0.87 and 1.00 in red. Some also capture JPEG compression noise, so the picture barely changes. I would choose $K = 5$: the remaining objective is small, and going to 10 colors adds little visible detail.

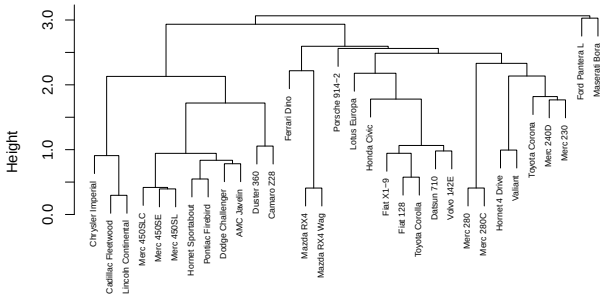
Question 3 Answers

Part a

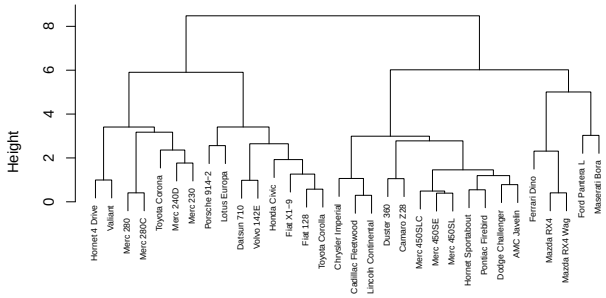
```
cars <- read.csv("data/mtcars.csv")
X_cars <- as.matrix(cars[, -1])
rownames(X_cars) <- cars$car
Z_cars <- scale(X_cars)
d_cars <- dist(Z_cars)

hc <- lapply(c(single = "single", complete = "complete", average = "average"),
             function(m) hclust(d_cars, method = m))

for (m in names(hc)) {
  par(mar = c(0.5, 4, 1, 0.5))
  plot(hc[[m]], cex = 0.55, xlab = "", sub = "", main = "", ylab = "Height")
}
```

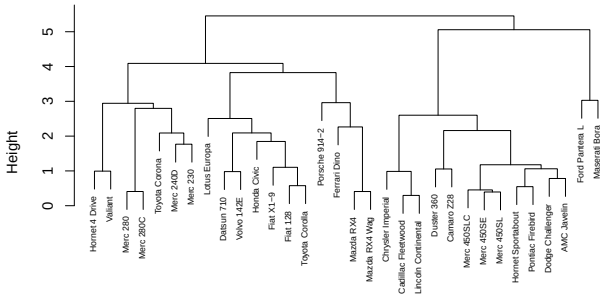


(a) Single linkage



(a) Complete linkage

Dendrograms of the standardized mtcars data under three linkages.



(a) Average linkage

Single linkage shows chaining. Cars join one at a time in a long staircase at similar heights, Ford Pantera L and Maserati Bora join last as an outlier pair, and no cut gives balanced groups. Complete linkage gives compact, balanced clusters and a clear structure. Its first split separates light economy cars from heavier and more powerful cars, which then divide into large V8 cars and a small group of sporty cars. Average linkage is in between. It agrees with complete linkage on the economy and V8 groups, but Ferrari Dino, the Mazda RX4s, and Porsche 914-2 join the economy branch, and Ford Pantera L and Maserati Bora stay separate until near the top. Close pairs, such as Merc 280/280C, Mazda RX4/RX4 Wag, and Cadillac Fleetwood/Lincoln Continental, merge at low heights under all three linkages.

Part b

```
k <- 3
groups <- cutree(hc$complete, k = k)

for (g in 1:k) cat("Cluster", g, ":", paste(names(groups)[groups == g], collapse = ", "), "\n\n")
```

```
Cluster 1 : Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino, Maserati Bora

Cluster 2 : Datsun 710, Hornet 4 Drive, Valiant, Merc 240D, Merc 230, Merc 280, Merc 280C, Fiat 128, Ho
9, Porsche 914-2, Lotus Europa, Volvo 142E

Cluster 3 : Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL, Merc 450SLC, Cadillac Fleetwood, Lin
```

```
cluster_means <- aggregate(X_cars, by = list(cluster = groups), FUN = mean)
cluster_means$n <- as.vector(table(groups))
print(round(cluster_means, 2))
```

```
  cluster   mpg  cyl   disp     hp drat   wt  qsec   vs   am gear carb  n
1       1 18.50 6.80 223.40 198.80 3.84 3.00 15.62 0.00 1.00 4.60 5.20  5
2       2 24.65 4.53 131.65  91.33 3.90 2.58 19.16 0.93 0.53 3.93 1.80 15
3       3 15.05 8.00 357.62 194.17 3.12 4.10 17.14 0.00 0.00 3.00 3.08 12
```

I use complete linkage with $K = 3$. In the dendrogram, the three groups are formed by height about 5 and stay separate until about 6, a long gap. Cutting into 4 clusters would only split the 5-car group into groups of 3 and 2.

- **Cluster 2: economy cars (15).** Highest mpg (24.7), mostly 4 cylinders, the lowest horsepower (91) and weight (2.58), and nearly all straight engines.
- **Cluster 3: large V8 cars (12).** All 8 cylinders, the largest displacement (358) and weight (4.10), the lowest mpg (15.1), and all automatic with 3 gears.
- **Cluster 1: sports cars (5).** Highest horsepower (199), fastest quarter-mile time (15.6 s), most carburetors (5.2), all manual with about 5 gears, and lighter than cluster 3.

Standardizing matters because the variables are on very different scales. Displacement ranges over hundreds and horsepower over hundreds, while `am` and `vs` are 0/1 and `wt` and `drat` vary by a few units. On the raw scale, Euclidean distance would be dominated by `disp` and `hp`, and the clusters would mostly reflect engine size. Standardizing gives every variable equal weight in the distance.